DCT-BASED IMAGE COMPRESSION AND QUALITY ANALYSIS

A PROJECT REPORT

Submitted by

HARSHIT [Reg No: RA2311003011776]

AKUL [Reg No: RA2311003011803]

BHAVYA [Reg No: RA2311003011878]

Under the Guidance of

Dr. Shoba LK

Assistant Professor, Department of Computing Technologies

in partial fulfillment of the requirements for the degree of

BACHELOR OF TECHNOLOGY

in

COMPUTER SCIENCE AND ENGINEERING



DEPARTMENT OF COMPUTING TECHNOLOGIES

COLLEGE OF ENGINEERING AND TECHNOLOGY

SRM INSTITUTE OF SCIENCE AND TECHNOLOGY

KATTANKULATHUR – 603 203

NOVEMBER 2026

SRM INSTITUTE OF SCIENCE & TECHNOLOGY
(Deemed to be University u/s 3 of UGC Act, 1956)

SRM INSTITUTE OF SCIENCE AND TECHNOLOGY

KATTANKULATHUR – 603 203

BONAFIDE CERTIFICATE

Certified that 21CSE251T Digital Image Processing project report titled "DCT-BASED IMAGE COMPRESSION AND QUALITY ANALYSIS" is the bonafide work of Harshit [Reg No: RA2311003011776], Akul [Reg No: RA2311003011803], and Bhavya [Reg No: RA2311003011878] who carried out the project work under my supervision. Certified further, that to the best of my knowledge, the work reported herein does not form part of any other thesis or dissertation on the basis of which a degree or award was conferred on an earlier occasion for this or any other candidate.

Dr. Shoba LK
SUPERVISOR
Assistant Professor
Department of Computing Technologies
SRMIST, Kattankulathur

Dr. G. NIRANJANA
HEAD OF THE DEPARTMENT
Professor
Department of Computing Technologies
SRMIST, Kattankulathur

SRM
INSTITUTE OF SCIENCE & TECHNOLOGY
(Deemed to be University u/s 3 of UGC Act, 1956)

Department of Computing Technologies

SRM Institute of Science and Technology — Own Work Declaration Form

Degree / Course	B.Tech in Computer Science and Engineering (21CSE251T — Digital Image Processing)
Student Names	Harshit, Akul, Bhavya
Registration Numbers	RA2311003011776, RA2311003011803, RA2311003011878
Title of Work	DCT-Based Image Compression and Quality Analysis

I/We hereby certify that this assessment complies with the University's Rules and Regulations relating to Academic Misconduct and Plagiarism, as listed in the University Website, Regulations, and the Education Committee guidelines. I/We confirm that all the work contained in this assessment is our own except where indicated, and that we have met the following conditions:

- Clearly referenced and listed all sources as appropriate, referenced and placed inverted commas around any quoted material from literature or technical specifications.
- Explicitly cited the origins of all diagrams, benchmark test blocks, equations, and empirical measurements incorporated in the report.
- Made no unauthorized use of the reports or submissions of any other student, either past or present, and acknowledged all technical assistance received.
- Complied fully with all academic integrity and originality requirements specified in the course syllabus and University examination handbook.
- Understand that any false claim regarding authorship for this academic submission will be penalized in accordance with University policies and regulations.

DECLARATION: I/We am/are aware of and understand the University's policy on Academic Misconduct and Plagiarism and certify that this assessment is our own work, except where indicated by referencing, and that we have followed the good academic practices noted above. Date: November 2026

Student 1 Signature:	Student 2 Signature:	Student 3 Signature:
Harshit (RA2311003011776)	Akul (RA2311003011803)	Bhavya (RA2311003011878)

ACKNOWLEDGEMENT

We express our humble gratitude to **Dr. C. Muthamizhchelvan**, Vice-Chancellor, SRM Institute of Science and Technology, for the world-class academic facilities extended for our project work and his continued encouragement.

We extend our sincere thanks to **Dr. M. Leenus J Martin**, Dean – College of Engineering and Technology, SRM Institute of Science and Technology, for his invaluable institutional support throughout our undergraduate program.

We wish to express our deep appreciation to **Dr. Revathi Venkataraman**, Professor and Chairperson, and **Dr. Pushpalatha**, Professor and Associate Chairperson, School of Computing, SRM Institute of Science and Technology, for their visionary guidance and continuous support throughout the course of this project.

We are incredibly grateful to our Head of the Department, **Dr. G. Niranjana**, Professor, Department of Computing Technologies, SRM Institute of Science and Technology, for her insightful suggestions, constructive feedback, and constant encouragement at all stages of the project work.

We register our immeasurable thanks to our Course Faculty and Project Supervisor, **Dr. Shoba LK**, Assistant Professor, Department of Computing Technologies, SRM Institute of Science and Technology, for her mentorship, technical guidance in Digital Image Processing (21CSE251T), and dedicated support in helping us design, verify, and complete this project.

Harshit [Reg No: RA2311003011776]
Akul [Reg No: RA2311003011803]
Bhavya [Reg No: RA2311003011878]

ABSTRACT

Digital images generate substantial storage and transmission overhead due to high spatial and spectral redundancy across neighboring pixels, necessitating lossy compression frameworks that balance bitrate reduction against perceptual fidelity. This project presents the design, implementation, and evaluation of a Baseline JPEG-style lossy image compression and quality analysis system built from scratch using the Two-Dimensional Type-II Discrete Cosine Transform (2-D DCT-II) in Python and NumPy, without relying on high-level frequency-transform libraries. Input grayscale or RGB images are converted via ITU-R BT.601 matrices with a +128 offset added to all three YCbCr components, followed by a −128 shift and partitioning into reflection-padded 8×8 blocks. A precomputed orthonormal DCT basis matrix concentrates signal energy into low-frequency coefficients, achieving 86.04% energy compaction in the single DC term on the Gonzalez & Woods sample block. Quantization uses the project's JPEG-style luminance table scaled dynamically across $Q \in [1, 100]$, with chroma quantized at Q_chroma = max(1, Q // 2). Following dequantization, inverse 2-D DCT, and reconstruction, the system computes Mean Squared Error (MSE), Peak Signal-to-Noise Ratio (PSNR), estimated Compression Ratio (Est. CR), and estimated Bits Per Pixel (Est. BPP) by counting 2 bytes per nonzero coefficient plus 1 byte per block without generating a binary bitstream. On the tested 256×256 grayscale and color images at Q = 50, the tool reports estimated compression ratios of 3.49× (31.72 dB) and 5.88× (28.27 dB), exceeding 30 dB PSNR at Q = 75. A Tkinter GUI, debounced slider preview, PNG/JPG export, CLI runner, 5-tab viva dialog, and 32 passing tests deliver an interactive academic laboratory.

TABLE OF CONTENTS

S.No / Section	Title	Page No.
	ABSTRACT	v
	LIST OF TABLES	vii
	LIST OF FIGURES	vii
	LIST OF SYMBOLS AND ABBREVIATIONS	viii
CHAPTER 1	**INTRODUCTION**	**1**
1.1	Background and Motivation	1
1.2	Problem Statement	2
1.3	Objectives of the Project	2
1.4	Scope and Technical Contributions	3
CHAPTER 2	**LITERATURE SURVEY AND THEORETICAL BACKGROUND**	**4**
2.1	Evolution of Image Compression Standards	4
2.2	Spatial, Statistical, and Psychovisual Redundancy	4
2.3	Comparative Analysis of Existing Compression Frameworks	5
CHAPTER 3	**SYSTEM ARCHITECTURE AND METHODOLOGY**	**6**
3.1	Overall System Architecture	6
3.2	End-to-End Signal Processing Pipeline Flowchart	6
3.3	Color Space Decomposition (ITU-R BT.601 +128 Offset Convention)	7
3.4	Channel Offset Shift, Block Partitioning, and Reflection Padding	8
3.5	Two-Dimensional Discrete Cosine Transform (DCT-II & IDCT-III)	8
3.6	Psychovisual Quantization and Quality Factor Scaling	9
3.7	Zig-Zag Scanning and Byte-Size Estimation	11
3.8	Objective Image Quality and Estimated Storage Metrics	12
3.9	Worked Numerical Walkthrough on Benchmark 8×8 Block (Theory Dialog Path)	12
CHAPTER 4	**IMPLEMENTATION AND SOFTWARE DESIGN**	**15**
4.1	Technology Stack and Software Environment	15
4.2	Modular Repository Design and Vectorization	15
4.3	Interactive Desktop GUI and Dual Export Design	16
4.4	Live 5-Tab Educational Theory & Calculation Pop-up	17
4.5	Command-Line Interface and Automated Test Suite	17
CHAPTER 5	**RESULTS AND DISCUSSION**	**19**
5.1	Visual Quality Evaluation Across Quality Factors	19
5.2	Quantitative Benchmark Results: Grayscale vs. Color Images	20
5.3	Rate–Distortion, Estimated Compression Ratio, and Sparsity Trends	21
5.4	Bitstream Estimate vs. File-Container Storage Discrepancy Analysis	23
CHAPTER 6	**CONCLUSION AND FUTURE SCOPE**	**25**
6.1	Conclusion	25
6.2	Limitations of the Current Implementation	25
6.3	Future Scope	25
	REFERENCES	27
APPENDIX A	**KEY SOURCE CODE EXCERPTS AND TEST LOG**	**28**

LIST OF TABLES

Table No.	Caption	Page No.
Table 2.1	Comparative Analysis of Image Storage Containers and Compression Methodologies	5
Table 3.1	Project's JPEG-Style Base Luminance Quantization Matrix (Q = 50)	10
Table 3.2	Scaled Quantization Matrix Behavior Across User Quality Settings	11
Table 3.3a	Input 8×8 Gonzalez & Woods Benchmark Pixel Block f(x, y)	12
Table 3.3b	Quantized Integer Coefficient Matrix F_Q(u, v) at Q = 50	13
Table 3.3c	Theory Dialog Reconstructed Pixels f_recon(x, y) via np.round() (Q = 50)	13
Table 4.1	Software Package Structure, Core Functions, and Module Responsibilities	15
Table 4.2	Automated Pytest Verification Suite Summary (32/32 Passing)	17
Table 5.1	Quantitative Performance on Tested Grayscale Image (assets/sample.png)	20
Table 5.2	Quantitative Performance on Tested Color Image (assets/sample_color.png)	20
Table 5.3	Comparison of Raw Memory, BMP, Heuristic Estimate, PNG, and JPG Sizes	23

LIST OF FIGURES

Figure No.	Caption	Page No.
Fig 1.1	Visual Overview: Uncompressed RGB vs. Q = 75 and Q = 10 (sample_color.png)	1
Fig 3.1	Modular Layered Software Architecture Diagram	6
Fig 3.2	End-to-End Signal Processing Pipeline Flowchart	7
Fig 3.3	All 64 Orthonormal 2-D DCT Basis Patterns (8×8 Spatial Grid)	9
Fig 3.4	Standard JPEG 8×8 Zig-Zag Scan Trajectory Matrix	11
Fig 3.5	Visual Heatmap Summary of Q = 50 Theory Dialog Walkthrough	14
Fig 4.1	Main Tkinter GUI Displaying Live Dual-Pane Compression & Metrics	16
Fig 4.2	Interactive 5-Tab Educational Theory & Calculation Pop-up Window	17
Fig 5.1	Grayscale 4-Up Comparison Grid (assets/sample.png)	19
Fig 5.2a	GUI-Exported 4-Up JPG Comparison Grid (avatar_comparison.jpg)	19
Fig 5.2b	GUI-Exported 4-Up PNG Comparison Grid (Cosmic Prince_comparison.png)	20
Fig 5.3	Measured PSNR (dB) vs. Quality Factor Q on Tested Sample Images	21
Fig 5.4	Estimated Compression Ratio (Est. CR) and Estimated Bitrate (Est. BPP) vs. Q	22
Fig 5.5	Quantized Coefficient Sparsity (% Zero Coefficients) vs. Q on Tested Images	22
Fig 5.6	PSNR vs. Estimated Bitrate (Est. BPP) Curve on Tested Sample Images	23

LIST OF SYMBOLS AND ABBREVIATIONS

Table 0.1: Notation, Mathematical Symbols, and Technical Abbreviations

Symbol / Acronym	Definition and Physical Meaning
DCT / IDCT	Discrete Cosine Transform (Type-II) / Inverse Discrete Cosine Transform (Type-III)
DIP	Digital Image Processing
JPEG	Joint Photographic Experts Group (ISO/IEC 10918-1 Standard)
PNG / BMP	Portable Network Graphics (Lossless) / Windows Bitmap (Uncompressed Raw)
HVS	Human Visual System
RGB	Red, Green, and Blue Additive Color Model
YCbCr	Luminance (Y), Blue-Difference Chroma (Cb), and Red-Difference Chroma (Cr)
ITU-R BT.601	International Telecommunication Union Recommendation for Standard Color Matrix
DC / AC	Direct Current (Zero-Frequency Average Coefficient F(0,0)) / Alternating Current Terms
Q / Q_chroma	User Quality Factor [1..100] for Luminance / Derived Chroma Quality max(1, Q // 2)
Q_50 / Q_scaled	Project's JPEG-style Base Luminance Table at Q=50 / Scaled Table for Quality Q
S(Q)	Standard libjpeg Quality Scaling Multiplier
D / D^T	Orthonormal 8×8 1-D DCT Basis Matrix and its Exact Matrix Transpose Inverse
f(x, y) / F(u, v)	Spatial Pixel Intensity Matrix and 2-D Frequency Coefficient Matrix
MSE / RMSE	Mean Squared Error / Root Mean Squared Error (Intensity Levels)
PSNR	Peak Signal-to-Noise Ratio measured in Decibels (dB)
Est. CR / Est. BPP	Estimated Compression Ratio (Raw Bytes / Est. Bytes) / Estimated Bits Per Pixel
RLE / EOB	Run-Length Style Byte Estimator (2B/nonzero + 1B EOB) / End-Of-Block Marker (0x00)
GUI / CLI	Graphical User Interface (Tkinter) / Command-Line Interface

CHAPTER 1
INTRODUCTION

1.1 Background and Motivation

Uncompressed digital images demand vast storage capacities and high transmission bandwidths because every spatial sample is stored independently in memory. A modest 256×256 full-color RGB photograph occupies 196,608 bytes (192.00 KB) in raw memory, while modern high-resolution sensors routinely capture tens of megabytes per frame. As articulated in Shannon's foundational information theory [1], efficient representation eliminates statistical redundancy without sacrificing essential signal semantics.

In natural imagery, adjacent pixels rarely fluctuate abruptly; smooth sky gradients, architectural surfaces, and organic textures display high spatial correlation [2]. Furthermore, the Human Visual System (HVS) exhibits asymmetrical sensitivity: viewers notice slight inaccuracies in overall brightness or coarse contours far more readily than subtle high-frequency variations or tiny hue shifts. Exploiting both statistical correlation and psychovisual insensitivity enables orders-of-magnitude reduction in data volume.

Since its standardization in 1991, the Joint Photographic Experts Group (JPEG) algorithm [3] has remained the most ubiquitous photographic compression pipeline worldwide. At its mathematical core lies the Two-Dimensional Discrete Cosine Transform (DCT), originally formulated by Ahmed, Natarajan, and Rao in 1974 [4]. Unlike complex Fourier transforms, the DCT operates strictly on real numbers and packs nearly all block signal energy into a few low-frequency coefficients.

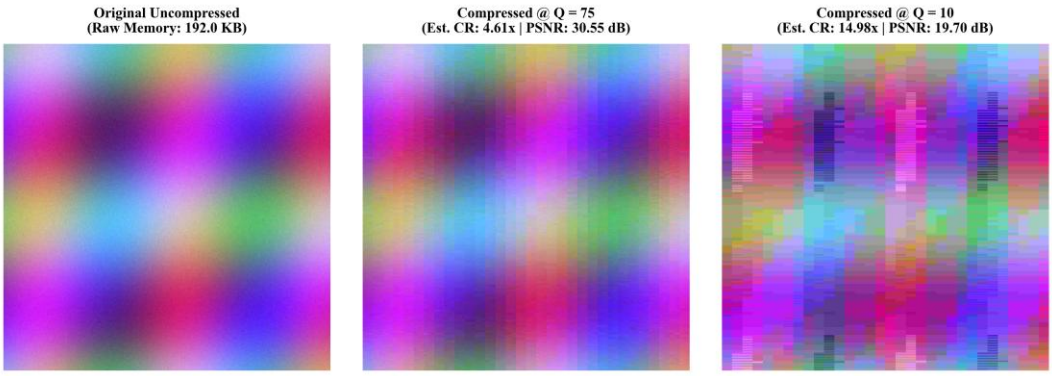


Fig 1.1: Visual comparison between uncompressed RGB pixel memory (192.0 KB) and reconstructed DCT outputs on sample_color.png at Q = 75 and Q = 10.

1.2 Problem Statement

Although everyday software tools seamlessly read and save compressed .jpg or .png files, commercial codecs operate as opaque black boxes. Students and practitioners rarely observe how individual 8×8 spatial blocks evolve inside the transform, how psychovisual quantization introduces sparsity, or why saving reconstructed pixels into different disk containers (such as PNG versus BMP) causes counterintuitive file-size measurements.

Existing rapid-prototyping libraries often wrap optimized C routines such as scipy.fft or libjpeg, obscuring the mathematical steps required in an undergraduate Digital Image Processing curriculum (21CSE251T). Consequently, there is a clear need for an open, transparent, from-scratch implementation that exposes every algebraic matrix operation while interactively quantifying the storage-versus-quality trade-off across both grayscale and color images.

1.3 Objectives of the Project

To bridge theoretical Digital Image Processing concepts with practical engineering verification, this project was designed around five primary objectives:

- **Transparent Mathematical Engine:** Implement the forward 2-D Type-II DCT, inverse 2-D Type-III DCT, 8×8 block partitioning with reflection padding, and JPEG-style quantization scaling purely in NumPy without calling black-box FFT/DCT libraries.
- **Perceptual Color Space Decomposition:** Develop an analytical ITU-R BT.601 RGB ↔ YCbCr color converter enabling independent quantization of luminance (Y at quality Q) and chrominance (Cb, Cr at Q_chroma = max(1, Q // 2)).
- **Quantitative Quality & Bitrate Analysis:** Compute Mean Squared Error (MSE), Peak Signal-to-Noise Ratio (PSNR), estimated Bits Per Pixel (Est. BPP), and estimated Compression Ratio (Est. CR) using a coefficient-counting byte estimator responsive to quality Q.
- **Interactive Visualization & Viva Defense Suite:** Build a responsive Tkinter desktop GUI featuring debounced slider previews, dual PNG/JPG export, 4-up comparison grids, and a dedicated 5-tab Flowchart, Algorithms & Numerical Calculations pop-up window.
- **Reproducibility & Automated Verification:** Provide a clean CLI tool, systematic multi-Q benchmark scripts, and an exhaustive automated unit and integration test suite using Pytest.

1.4 Scope and Technical Contributions

1.4.1 ARBITRARY RESOLUTION SUPPORT AND VECTORIZATION

The system accepts arbitrary image dimensions in PNG, JPG, BMP, or TIFF formats, automatically handling non-multiples of 8 through edge reflection padding prior to block splitting and symmetrical cropping upon merging. Performance remains interactive even on megapixel photography because all N blocks per channel are transformed simultaneously in vectorized NumPy einsum tensor batches.

1.4.2 INTEGRATED VIVA AND CALCULATION ARCHITECTURE

A distinguishing contribution of this project is its integrated **Educational Core (`dct_core/explanation.py`)** and **Interactive Theory Dialog (`app/theory_dialog.py`)**. Rather than displaying only final PSNR numbers, the software computes live 8×8 matrix intermediates—including exact DC energy compaction percentage, coefficient sparsity, zig-zag sequences, and run-length (run, value) tuples—and exports complete academic derivations.

CHAPTER 2
LITERATURE SURVEY AND THEORETICAL BACKGROUND

2.1 Evolution of Image Compression Standards

Data compression rests upon two distinct mathematical paradigms: lossless entropy encoding and lossy transform coding. Early variable-length prefix coding introduced by Huffman [5] achieved optimal symbol-level lossless compression when source probabilities were known. However, operating directly on raw spatial pixels ignores strong two-dimensional neighborhood correlations, limiting lossless compression ratios on natural photographs to roughly 1.5:1–2.5:1.

Theoretical analysis established that the Karhunen–Loève Transform (KLT) completely decorrelates stationary Markov signals and maximizes energy compaction [6]. Unfortunately, the KLT basis depends on data covariance and lacks a fast transform algorithm. In 1974, Ahmed, Natarajan, and Rao [4] proved that the Discrete Cosine Transform closely approximates KLT energy compaction for highly correlated signals while utilizing a fixed, image-independent basis.

Building on two decades of transform research, the Joint Photographic Experts Group published ISO/IEC 10918-1 [7], thoroughly documented by Wallace [3] and Pennebaker and Mitchell [8]. Baseline JPEG standardized 8×8 block splitting, the 2-D DCT, empirically derived luminance/chrominance quantization tables tuned for human visual contrast, zig-zag reordering, run-length zero encoding, and Huffman entropy coding into a unified international format.

2.2 Spatial, Statistical, and Psychovisual Redundancy

Digital image compression targets three fundamental forms of data redundancy described by Gonzalez and Woods [2]. First, *coding redundancy* arises when fixed-length 8-bit integers are assigned equally to frequent and rare intensity values. Second, *spatial redundancy* occurs because neighboring pixel values in smooth regions or repeated textures can be predicted from their surroundings.

Third, and uniquely critical to lossy codecs, *psychovisual redundancy* exploits physiological limitations in human retinal photoreceptors. Mannos and Sakrison [9] characterized the Modulation Transfer Function (MTF) of human vision, demonstrating that contrast sensitivity peaks around medium spatial frequencies and drops sharply for fine high-frequency patterns. Dividing high-frequency DCT coefficients by large quantization steps discards noise imperceptible to viewers.

Similarly, human retinas contain vastly more brightness-sensitive rod cells than color-sensitive cone cells. As codified in ITU-R Recommendation BT.601 [10], transforming RGB primaries into Luminance (Y) and Chrominance (Cb, Cr) channels isolates detail-bearing intensity from slowly varying color, permitting heavier quantization on chroma planes without inducing objectionable subjective degradation.

2.3 Comparative Analysis of Existing Compression Frameworks

Contemporary imaging ecosystems employ diverse compression standards tailored to specific fidelity, complexity, and licensing constraints. Portable Network Graphics (PNG) [11] utilizes spatial line-filtering paired with LZ77/Huffman DEFLATE compression to guarantee bit-exact lossless preservation for diagrams, screenshots, and transparency masks, but produces substantially larger files than JPEG on noisy photographic scenes.

Later architectures sought to overcome JPEG's 8×8 block-boundary artifacts. JPEG 2000 [12] replaced localized 8×8 DCT blocks with global Discrete Wavelet Transforms (DWT) and embedded EBCOT bit-plane coding, yielding superior rate-distortion behavior at extreme compression at the cost of significantly higher memory and CPU demands. Google's WebP standard [13] adapted VP8 intra-frame directional prediction alongside 4×4 transforms.

Evaluating distortion across these codecs traditionally relies on Pixel-wise Mean Squared Error (MSE) and Peak Signal-to-Noise Ratio (PSNR). Although newer perceptual structural metrics such as SSIM [14] model local luminance and contrast masking, Huynh-Thu and Ghanbari [15] confirmed that PSNR remains a rigorous, reproducible benchmark when comparing controlled quantization variations on a fixed source image and codec architecture.

Table 2.1: Comparative Analysis of Image Storage Containers and Compression Methodologies

Standard / Format	Core Transform / Method	Nature	Strengths	Key Limitations
BMP (Bitmap)	Raw Uncompressed Raster	Lossless	Zero decode latency; exact pixel array representation	Massive file size; zero compression savings
PNG (RFC 2083) [11]	Scanline Filtering + DEFLATE	Lossless	Bit-exact fidelity; sharp text/UI lines; alpha support	Poor compression on natural continuous-tone photos
Baseline JPEG [3, 7]	8×8 2-D DCT + Quantization + RLE/Huffman	Lossy	Fast $O(N^3)$ block math; high CR (5×–15×); universal	8×8 blocking and ringing artifacts at very low Q
JPEG 2000 [12]	Full-Frame 2-D DWT + EBCOT	Both	No block boundaries; progressive resolution scalability	High computational/RAM cost; weak web browser adoption
WebP [13]	VP8 Intra-Prediction + 4×4 DCT/WHT	Both	25–34% smaller than JPEG at matched SSIM; alpha	Higher encoder complexity than baseline JPEG
Our DIP System	Vectorized 8×8 DCT-II + BT.601 + RLE Est.	Lossy (Demo)	Transparent intermediate matrices; live Q & viva walkthrough	Uses teaching RLE estimator rather than Huffman bitstream

CHAPTER 3
SYSTEM ARCHITECTURE AND METHODOLOGY

3.1 Overall System Architecture

The software system follows a clean modular architecture separating user presentation layers from the underlying signal-processing pipeline and mathematical primitives. This strict separation of concerns ensures that the exact same verified mathematical engine drives the graphical interface, CLI tool, automated benchmarks, and unit test suite.

As illustrated in Fig 3.1, requests originate from either the Tkinter GUI (app/gui.py) or CLI runner (main.py), enter the orchestration layer (pipeline/pipeline.py), invoke orthonormal transforms and scaling tables inside dct_core/, and conclude with objective distortion and bitstream size measurements in quality/metrics.py.

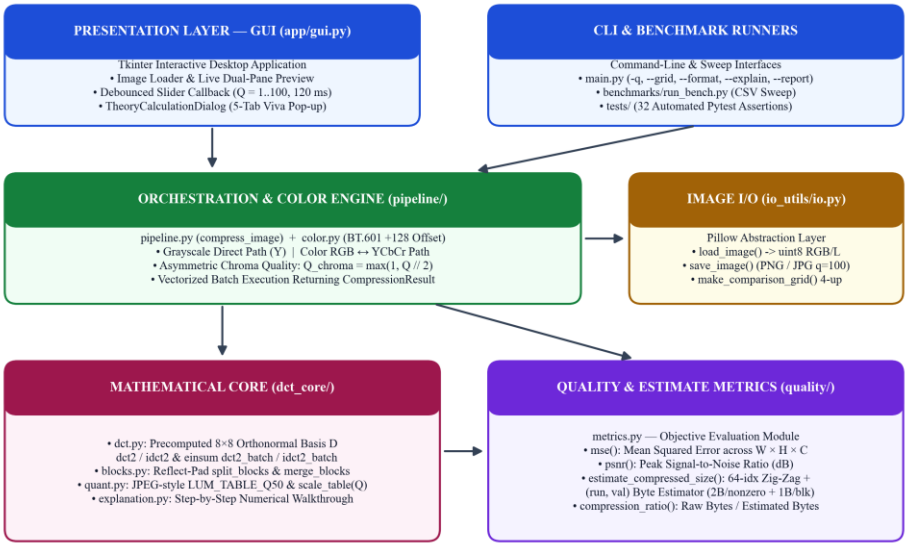


Fig 3.1: Modular layered software architecture separating Presentation (GUI/CLI), Pipeline Orchestration, Mathematical Core, Quality Metrics, and Image I/O.

3.2 End-to-End Signal Processing Pipeline Flowchart

Compression and reconstruction execute sequentially through a deterministic ten-step signal-processing chain modeled after baseline JPEG [3, 7]. Color inputs branch into independent luminance and chrominance channels, undergo encoding (level shift, 8×8 tiling, forward DCT, psychovisual quantization), branch simultaneously to the bitstream estimator and symmetric decoder, and recombine for evaluation.

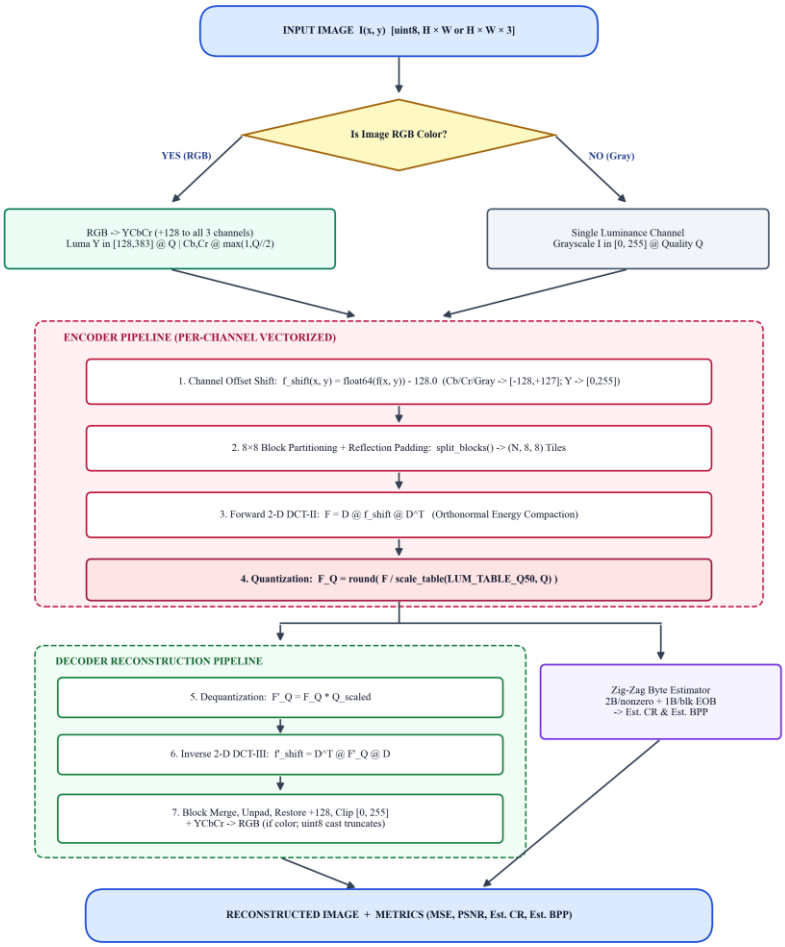


Fig 3.2: Complete end-to-end signal processing pipeline flowchart showing color routing, encoder compression stages, zig-zag size estimation, and decoder reconstruction.

3.3 Color Space Decomposition (ITU-R BT.601 +128 Offset Convention)

When an input image contains three RGB color planes ((H, W, 3)), compressing red, green, and blue channels directly is suboptimal because RGB planes remain heavily correlated with brightness variations. In pipeline/color.py, rgb_to_ycbcr() applies the ITU-R BT.601 [10] conversion matrix in floating point (float64) and adds a uniform +128.0 offset to **all three** components (Y, Cb, and Cr), rather than adding +128 solely to chrominance:

$$Y = 0.299000 \cdot R + 0.587000 \cdot G + 0.114000 \cdot B + 128 \tag{3.1}$$

$$Cb = -0.168736 \cdot R - 0.331264 \cdot G + 0.500000 \cdot B + 128 \tag{3.2}$$

$$Cr = 0.500000 \cdot R - 0.418688 \cdot G - 0.081312 \cdot B + 128 \tag{3.3}$$

Under this uniform +128 convention for R, G, B ∈ [0, 255], Cb and Cr span approximately [0.0, 255.0] whereas luminance Y spans [128.0, 383.0] (for example, Y ∈ [177.91, 334.38] on sample_color.png). Notice that because _compress_channel() unconditionally subtracts 128.0 before the 2-D DCT and restores +128.0 afterward, this initial luma offset cancels prior to transformation. On inverse reconstruction, ycbcr_to_rgb() subtracts 128.0 from all three channels before multiplying by the inverse BT.601 matrix and clipping to uint8 [0, 255]:

$$R = (Y-128) + 1.402\cdot(Cr-128),\ \ G = (Y-128) - 0.344136\cdot(Cb-128) - 0.714136\cdot(Cr-128),\ \ B = (Y-128) + 1.772\cdot(Cb-128) \tag{3.4}$$

Due to finite 4-decimal matrix constants and floating-point-to-uint8 truncation, test_roundtrip_rgb() in tests/test_color.py verifies the unquantized RGB ↔ YCbCr round-trip as **accurate within ±1 sample unit**. During compression, Y uses user quality Q while Cb and Cr use Q_chroma = max(1, Q // 2).

3.4 Channel Offset Shift, Block Partitioning, and Reflection Padding

Before computing frequency transforms, _compress_channel() casts each input plane to float64 and subtracts 128.0. For grayscale images ([0, 255]) and color Cb/Cr planes ([0, 255]), this centers inputs to [−128, +127] as in classic JPEG; for color Y (which carries the +128 offset from rgb_to_ycbcr()), subtracting 128 maps Y back into [0, 255] before entering the DCT.

Next, dct_core/blocks.py divides each 2-D channel (H, W) into non-overlapping 8×8 pixel tiles. Whenever height H or width W is not an exact multiple of 8, split_blocks() applies symmetric reflection padding (numpy.pad(..., mode='reflect')) to reach (Hp, Wp). Unlike zero padding—which introduces harsh artificial black borders that trigger high-frequency ringing—reflection preserves local edge continuity and is cleanly cropped back to (H, W) after reconstruction.

3.5 Two-Dimensional Discrete Cosine Transform (DCT-II & IDCT-III)

3.5.1 SEPARABLE ORTHONORMAL MATRIX FORMULATION

Each zero-centered 8×8 spatial block f_shift(x, y) is projected onto 64 orthogonal cosine basis waveforms via the forward Two-Dimensional Type-II DCT [4, 6]:

$$F(u, v) = \alpha(u)\cdot\alpha(v) \cdot \sum_{x=0..7} \sum_{y=0..7} f_shift(x, y) \cdot \cos[(2x + 1)u\pi / 16] \cdot \cos[(2y + 1)v\pi / 16] \tag{3.5}$$

Here spatial indices x, y ∈ {0..7} and frequency indices u, v ∈ {0..7} utilize the orthonormal scale function α(0) = 1 / √8 and α(k > 0) = √(2 / 8) = 0.5. Because the 2-D transform kernel is separable along rows and columns, direct matrix evaluation replaces expensive $O(N^4)$ nested loops with two $O(N^3)$ matrix products using a precomputed 8×8 matrix D:

$$D[k, i] = \alpha(k) \cdot \cos[(2i + 1)k\pi / 16], \quad F = D \cdot f_shift \cdot D\text{^}T, \quad f_recon = D\text{^}T \cdot F'_Q \cdot D \tag{3.6}$$

Because D is strictly orthonormal (D · D^T = I_8, verified to $< 10^{-10}$ tolerance in our unit tests), the inverse transform Type-III IDCT requires only matrix transposition (D^T). Fig 3.3 visualizes all 64 spatial frequency patterns encoded by D, ranging from the constant flat average DC basis at (0, 0) to rapid diagonal oscillations at (7, 7).

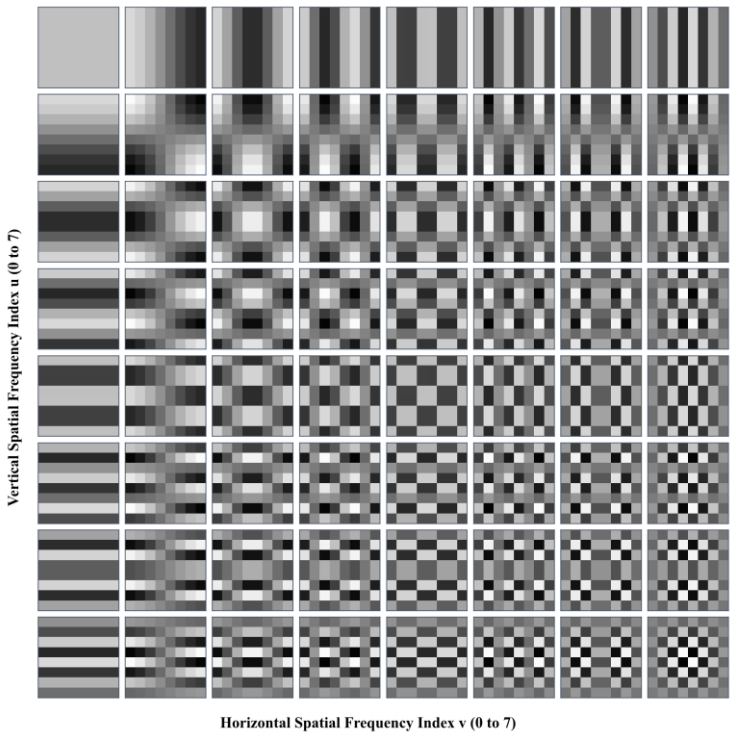


Fig 3.3: All 64 orthonormal 2-D DCT basis patterns generated directly from matrix D, arranged from DC (0,0) at top-left to maximum diagonal frequency (7,7) at bottom-right.

3.6 Psychovisual Quantization and Quality Factor Scaling

The Discrete Cosine Transform itself is completely reversible and achieves zero size reduction on its own. Compression occurs solely during **psychovisual quantization**, where transform coefficients F(u, v) are divided element-wise by a perceptual threshold matrix and rounded to the nearest integer. Low frequencies near (0, 0) receive small divisors to preserve contrast accuracy, whereas bottom-right diagonal frequencies receive divisors exceeding 100.

Table 3.1: Project's JPEG-Style Base Luminance Quantization Matrix (LUM_TABLE_Q50 at Q = 50)

u \ v	0	1	2	3	4	5	6	7
0	16	11	10	16	24	40	51	61
1	12	12	14	19	26	58	60	55
2	14	13	16	24	40	57	69	56
3	14	17	22	29	51	87	80	62
4	18	22	37	56	68	103	77	62
5	30	35	55	64	81	104	92	75
6	50	72	78	87	103	121	100	78
7	68	92	95	98	112	100	103	99

While upper-left entries of LUM_TABLE_Q50 match the classic ITU-T T.81 / ISO/IEC 10918-1 [7] Annex K.1 table, several lower-right entries differ (for instance, [103, 77, 62] vs. [109, 103, 77] in row 4, and [68] vs. [72] at (7, 0)). Accordingly, Table 3.1 represents the project's JPEG-style luminance table. To scale steps across Q ∈ [1, 100], dct_core/quant.py implements the Independent JPEG Group (libjpeg) scaling law [16]:

$$S(Q) = 5000 / Q \ \ (\text{if } Q < 50), \quad \text{or} \quad 200 - 2 \cdot Q \ \ (\text{if } Q \geq 50) \tag{3.7}$$

$$Q_scaled(u, v) = clip(floor[(Q_50(u, v) \cdot S(Q) + 50) / 100], 1, 255) \tag{3.8}$$

$$F_Q(u, v) = round(F(u, v) / Q_scaled(u, v)), \qquad F'_Q(u, v) = F_Q(u, v) \cdot Q_scaled(u, v) \tag{3.9}$$

At Q = 100, S(100) = 0, clamping every divisor in Q_scaled to 1. However, Q = 100 is **not** strictly lossless: quantization still rounds floating-point DCT coefficients to integers (np.round) and the pipeline truncates float64 samples when casting to uint8, leaving a nonzero grayscale MSE (0.497 in Table 5.1); moreover, RGB mode still quantizes Cb and Cr at Q_chroma = 50 (MSE = 35.301 in Table 5.2). Conversely, at Q = 10, S(10) = 500 clamps most high-frequency divisors to 255, forcing faint AC terms to zero.

Table 3.2: Scaled Quantization Matrix Behavior Across User Quality Settings

Quality Setting	Multiplier S(Q)	DC Step (0,0)	Mid-Freq Step (3,3)	High-Freq Step (7,7)	Quantization Behavior
Q = 10 (Low)	500.0	80	145	255 (clamped)	Aggressive zeroing; strong 8×8 blocking artifacts
Q = 50 (Reference)	100.0	16	29	99	Project's JPEG-style Q=50 luminance table (Table 3.1)
Q = 75 (Default)	50.0	8	15	50	Finer divisors; chroma uses Q_chroma = 37 (S = 135.1)
Q = 90 (High)	20.0	3	6	20	Small divisors; slight residual error (chroma at Q = 45)
Q = 100 (Max)	0.0	1	1	1	Unit step rounding (nonzero MSE; RGB chroma uses Q = 50)

3.7 Zig-Zag Scanning and Byte-Size Estimation

After quantization, nonzero integers cluster tightly in the upper-left corner of each 8×8 matrix F_Q, leaving the lower-right triangle populated mostly by zeros. Scanning row-by-row would interleave active low-frequency terms with zeros. As shown in Fig 3.4, diagonal zig-zag traversal [3, 7] orders all 64 coefficients strictly from low to high spatial frequency, grouping trailing zeros contiguously.

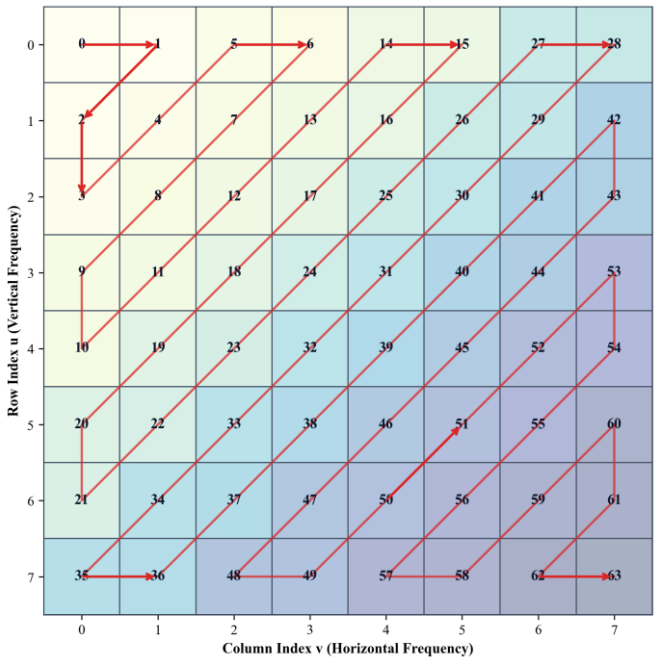


Fig 3.4: Standard JPEG 8×8 zig-zag scan trajectory mapping 2-D spectral coordinates (u, v) to 1-D sequential indices 0 through 63.

In quality/metrics.py, estimate_compressed_size() walks this 64-element zig-zag vector per block and **estimates** compressed size by counting 2 bytes per nonzero coefficient (representing a (run, value) pair) plus 1 byte per block for an EOB marker, without writing any actual binary or Huffman-

coded bitstream. Consequently, all Compression Ratio and Bits Per Pixel figures derived from it represent **estimated** rather than bitstream-encoded sizes.

3.8 Objective Image Quality and Estimated Storage Metrics

To quantify reconstruction distortion and estimated storage trade-offs, quality/metrics.py computes Mean Squared Error (MSE), Peak Signal-to-Noise Ratio (PSNR), **estimated Compression Ratio (Est. CR)**, and **estimated Bits Per Pixel (Est. BPP)** over all W × H × C scalar samples comparing original I against reconstructed I_recon [2, 15]:

$$MSE = [\ 1 / (W \cdot H \cdot C)\ ] \cdot \sum_{x} \sum_{y} \sum_{c} [\ I(x, y, c) - I_recon(x, y, c)\ ]^2 \quad (3.10)$$

$$PSNR = 10 \cdot \log_{10}(255^2 / MSE\ ) \ dB \quad [defined\ as\ +\infty\ dB\ when\ MSE = 0] \quad (3.11)$$

$$Est.\ CR = Raw\ Bytes\ (W \cdot H \cdot C) / Estimated\ Bytes, \qquad Est.\ BPP = (Estimated\ Bytes \times 8) / (W \cdot H) \quad (3.12)$$

3.9 Worked Numerical Walkthrough on Benchmark 8×8 Block (Theory Dialog Path)

3.9.1 WALKTHROUGH INTERMEDIATES AND PIPELINE ROUNDING DIFFERENCE

Tables 3.3a–c and Fig 3.5 present the worked Q = 50 example generated by the **interactive theory dialog path** (dct_core/explanation.py::calculate_block_steps) on the classic Gonzalez & Woods [2] 8×8 test patch. After subtracting 128, forward 2-D DCT concentrates **86.04%** of total block energy in the single DC coefficient F(0, 0) = −415.37 and **94.54%** inside the top-left 3×3 sub-block.

Table 3.3a: Input 8×8 Gonzalez & Woods Benchmark Pixel Block f(x, y)

u \ v	0	1	2	3	4	5	6	7
0	52	55	61	66	70	61	64	73
1	63	59	55	90	109	85	69	72
2	62	59	68	113	144	104	66	73
3	63	58	71	122	154	106	70	69
4	67	61	68	104	126	88	68	70
5	79	65	60	70	77	68	58	75
6	85	71	64	59	55	61	65	83
7	87	79	69	68	65	76	78	94

Table 3.3b: Quantized Integer Coefficient Matrix F_Q(u, v) at Q = 50 (44/64 = 68.8% Exact Zeros)

u \ v	0	1	2	3	4	5	6	7
0	-26	-3	-6	2	2	-1	0	0
1	0	-2	-4	1	1	0	0	0
2	-3	1	5	-1	-1	0	0	0
3	-3	1	2	-1	0	0	0	0
4	1	0	0	0	0	0	0	0
5	0	0	0	0	0	0	0	0
6	0	0	0	0	0	0	0	0
7	0	0	0	0	0	0	0	0

Table 3.3c: Theory Dialog Reconstructed Pixels f_recon(x, y) via np.round() (MSE = 35.5625, PSNR = 32.62 dB)

u \ v	0	1	2	3	4	5	6	7
0	62	65	57	60	72	63	60	82
1	57	55	56	82	108	87	62	71
2	58	50	60	111	148	114	67	65
3	65	55	66	120	155	114	68	70
4	70	63	67	101	122	88	60	78
5	71	71	64	70	80	62	56	81
6	75	82	67	54	63	65	66	83
7	81	94	75	54	68	81	81	87

Dividing by LUM_TABLE_Q50 forces 44 of 64 entries (68.8% sparsity) to zero, yielding 41 estimated bytes (Est. CR = 1.56×). Note a subtle implementation distinction between code paths: calculate_block_steps() (Theory Dialog) applies np.round(rec_shifted + 128.0) before casting to uint8 (MSE = 35.5625, PSNR = 32.62 dB), whereas the **main grayscale pipeline** (pipeline/pipeline.py::compress_image) truncates floating-point values directly when casting .astype(np.uint8) without np.round(), producing MSE = 36.2188 and PSNR = 32.54 dB on the exact same 8×8 block.

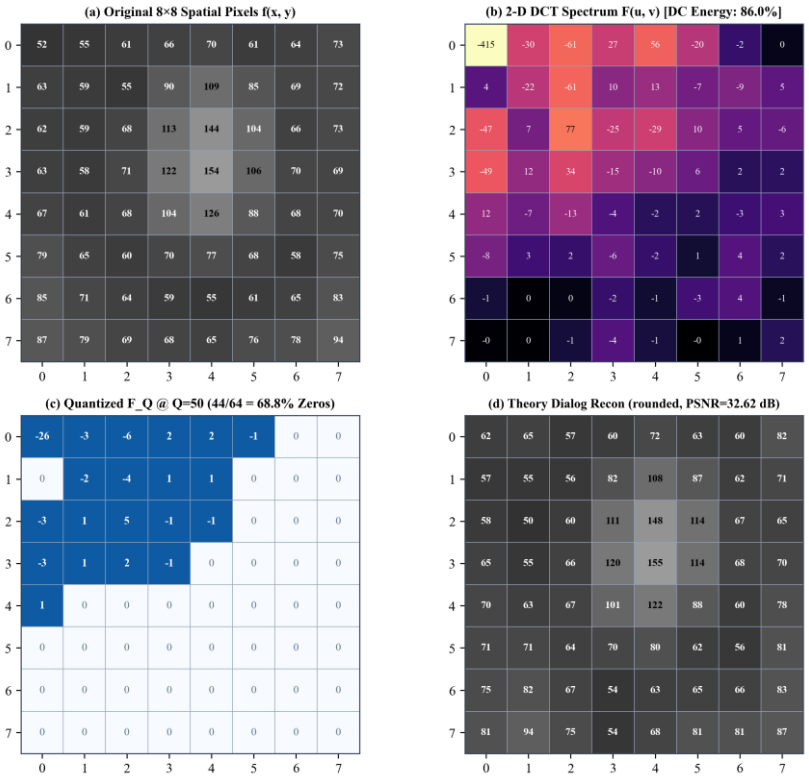


Fig 3.5: Visual heatmap summary of the Q = 50 Theory Dialog walkthrough (calculate_block_steps, rounded to uint8) on the 8×8 benchmark block.

CHAPTER 4
IMPLEMENTATION AND SOFTWARE DESIGN

4.1 Technology Stack and Software Environment

The application is authored in **Python 3.13** with strict reliance on open-source scientific computing libraries. **NumPy** [17] provides double-precision (float64) matrix multiplication and batch einsum tensor contractions, eliminating Python pixel loops so full 256×256 RGB compression completes in milliseconds. **Pillow (PIL Fork)** [18] handles image file decoding, color mode normalization, and comparison montage rendering.

Graphical presentation uses Python's native **Tkinter / ttk** widget toolkit with the clam theme, ensuring zero external GUI installation requirements across Windows, macOS, and Linux desktops. Automated behavioral verification is implemented with **pytest 8.3.4** covering mathematical invariants, boundary padding, GUI lifecycle, and CLI reporting.

4.2 Modular Repository Design and Vectorization

Table 4.1: Software Package Structure, Core Functions, and Module Responsibilities

Package / File	Key Symbols & Entry Points	Primary Architectural Responsibility
dct_core/dct.py	D, dct2(), idct2(), dct2_batch(), idct2_batch()	Precomputes 8×8 orthonormal DCT basis D; executes vectorized einsum('ij,njk,kl->nil', ...) batch transforms
dct_core/blocks.py	split_blocks(), merge_blocks()	Applies reflection edge padding for arbitrary sizes, reshapes 2-D planes into (N, 8, 8) tiles, and unpads output
dct_core/quant.py	LUM_TABLE_Q50, scale_table(), quantize(), dequantize()	Encodes project's JPEG-style Q=50 luminance table, computes libjpeg S(Q) multiplier, and rounds coefficients
dct_core/explanation.py	calculate_block_steps(), get_pipeline_flowchart()	Produces ASCII flowcharts, formal pseudocode, equations, live 8×8 numerical intermediates, and reports
pipeline/color.py	rgb_to_ycbcr(), ycbcr_to_rgb()	BT.601 matrices adding +128 to all 3 channels; round-trip verified within ±1 sample unit
pipeline/pipeline.py	compress_image(), CompressionResult	Detects grayscale vs RGB, assigns Q and Q_chroma = max(1, Q//2), orchestrates pipeline & metrics
quality/metrics.py	mse(), psnr(), estimate_compressed_size(), compression_ratio()	Computes distortion scores and estimated size (2B per nonzero coeff + 1B EOB per block)
io_utils/io.py	load_image(), save_image(), make_comparison_grid()	Image reading/writing (lossless PNG or second-encode JPG q=100) and 4-up montage generation
app/gui.py & theory_dialog.py	DCTApp, TheoryCalculationDialog	Main desktop window (120 ms debounced Tkinter callback) and 5-tab educational viva window
main.py & run_bench.py	main(), run()	CLI single-image processor (--grid, --format, --explain, --report) and CSV benchmark sweep tool

4.2.1 VECTORIZED EINSUM TENSOR BATCHING

Executing dct2() sequentially inside Python for loops across thousands of 8×8 blocks introduces substantial interpreter overhead. By reshaping each padded channel into a 3-D NumPy tensor (N, 8, 8) inside split_blocks(), dct2_batch() contracts D, all N image blocks, and D^T simultaneously via a single C-backed np.einsum call, keeping interactive slider dragging smooth on standard laptop hardware.

4.3 Interactive Desktop GUI and Dual Export Design

Launched via python app/gui.py, the main desktop interface (DCTApp) displays the original uncompressed image on the left and the reconstructed output on the right (360×360 Lanczos-scaled previews). Dragging the horizontal quality slider across Q = 1..100 schedules a lightweight 120 ms debounced Tkinter event-loop callback (self.root.after) on the main GUI thread so continuous mouse drags remain responsive without recomputing every intermediate pixel tick.

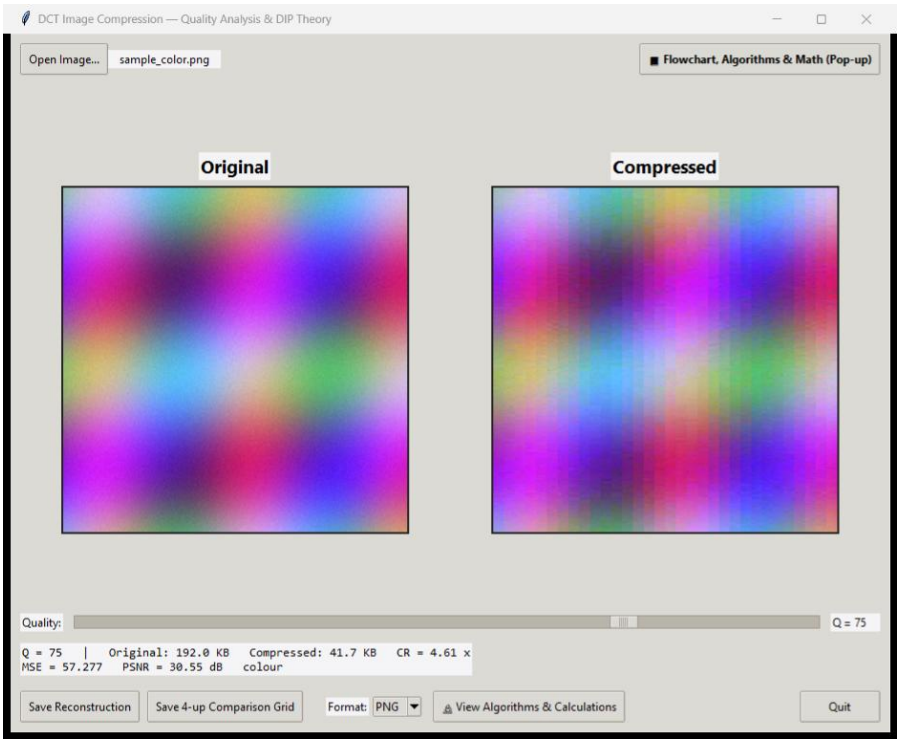


Fig 4.1: Main Tkinter GUI displaying Original vs. Compressed (Q = 75) preview on sample_color.png, live objective metrics readout, Format dropdown (PNG/JPG), and Educational Pop-up controls.

A dedicated Format: read-only dropdown (PNG or JPG) lets users export single reconstructions (*_recon_q<Q>.<ext>) or 4-up comparison grids (Original | Q = 75 | Q = 50 | Q = 10 in GUI; Original | Q = 90 | Q = 50 | Q = 10 in CLI). Exporting .png saves the exact reconstructed array losslessly for faithful artifact inspection, whereas exporting .jpg triggers Pillow's JPEG encoder at quality=100—which performs a **second lossy encode** (re-applying JPEG color conversion, default 4:2:0

subsampling, DCT, and quantization). Saving the same sample_color.png (Q = 75) reconstruction as .png and .jpg changed 163,546 of 196,608 RGB channel values.

4.4 Live 5-Tab Educational Theory & Calculation Pop-up

Designed explicitly for classroom demonstrations and oral viva evaluation, clicking `**[Flowchart, Algorithms & Math]**` opens TheoryCalculationDialog synchronized live with the main window's active image and slider quality Q. Its five tabs present the Pipeline Flowchart, Formal Pseudocode, Governing Equations, an interactive 8×8 Numerical Walkthrough (switchable between the Gonzalez & Woods block and actual blocks extracted from the loaded photo), and full Global Metrics derivations—plus a one-click .txt academic report exporter.

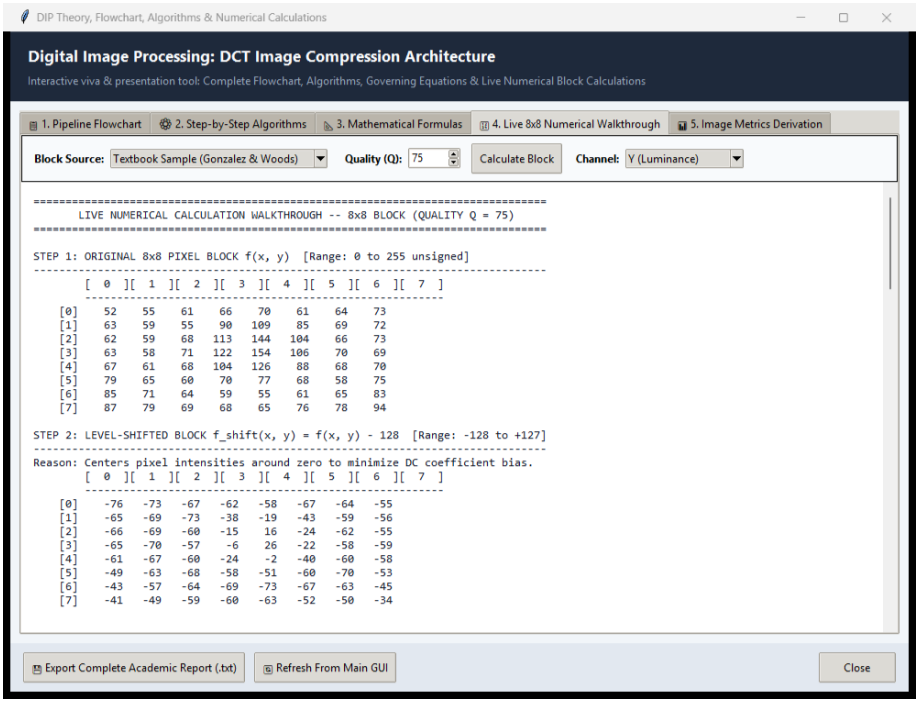


Fig 4.2: Interactive 5-tab DIP Theory & Numerical Calculation Pop-up Window (Tab 4 shown) displaying live 8×8 step-by-step matrices and academic export controls.

4.5 Command-Line Interface and Automated Test Suite

For batch scripting and headless verification, main.py exposes flags -q <1..100>, --no-color, --grid, --format {png,jpg}, --explain (terminal walkthrough printout), and --report <file.txt>. Similarly, benchmarks/run_bench.py sweeps arbitrary quality lists and logs structured CSV telemetry. All components pass an automated pytest suite comprising **32 tests across 7 modules** in 16.86 seconds with zero skipped assertions.

Table 4.2: Automated Pytest Verification Suite Summary (Total: 32/32 Tests Passing)

Test Module	Test Count	Verified Invariants & Edge Cases	Status

Test Module	Test Count	Verified Invariants & Edge Cases	Status
tests/test_dct.py	5	D @ D.T == I_8 (<1e-10); single & batch round-trip (<1e-9); DC-only constant check; shape errors	5 PASSED
tests/test_quant.py	5	Q=100 unit step; Q=1 extreme scaling; Q=50 identity match; rounding error <= step/2; bounds check	5 PASSED
tests/test_metrics.py	5	Zero MSE & +inf PSNR for identical arrays; Gaussian noise sigma=10 check (~28.13 dB); RLE size sensitivity	5 PASSED
tests/test_color.py	3	BT.601 RGB <-> YCbCr round-trip within ±1; RGB compression & Q_chroma = Q//2; 3-channel raw byte count	3 PASSED
tests/test_pipeline.py	3	Smooth gradient PSNR > 35 dB at Q=75; non-multiple-of-8 (37×53) reflect pad/crop; Q=100 within ±1	3 PASSED
tests/test_io.py	3	Lossless grayscale & RGB PNG round-trip equality; 4-up comparison montage creation on disk	3 PASSED
tests/test_explanation.py	8	Textbook DC=-415.38 & F_Q(0,0)=-26 at Q=50; Q=10 vs Q=95 monotonicity; CLI flags & GUI dialog sync	8 PASSED

CHAPTER 5
RESULTS AND DISCUSSION

5.1 Visual Quality Evaluation Across Quality Factors

5.1.1 BLOCKING AND CHROMINANCE ARTIFACT OBSERVATIONS

Visual inspection across progressively lower quality settings reveals the classic perceptual hallmarks of block-DCT compression. On smooth grayscale luminance gradients (assets/sample.png, Fig 5.1), reconstructions at Q = 90 (37.19 dB) remain practically indistinguishable from the original. At Q = 50 (31.72 dB), mild contour banding becomes noticeable upon close inspection, whereas Q = 10 (26.42 dB) exposes unmistakable 8×8 tile discontinuities (blocking artifacts).

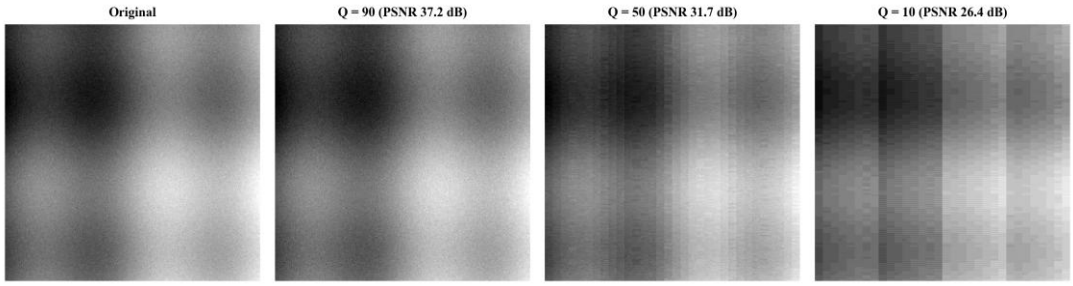


Fig 5.1: Grayscale 4-up reconstruction comparison on assets/sample.png (256×256) comparing Original against Q = 90 (37.19 dB), Q = 50 (31.72 dB), and Q = 10 (26.42 dB).

On the complex RGB illustrations (avatar_comparison.jpg and Cosmic Prince_comparison.png, Fig 5.2), setting Q_chroma = max(1, 10 // 2) = 5 at Q = 10 (S(5) = 1000.0) heavily quantizes chrominance alongside luminance. Color gradients collapse into coarse grayscale-like blocks and magenta/cyan blotches, whereas Q = 75 retains fine edge lighting and vivid color reproduction.



Fig 5.2a: GUI-exported 4-up JPG comparison grid on avatar.jpg (Original, Q = 75, Q = 50, and Q = 10) illustrating chromatic breakdown at extreme compression.



Fig 5.2b: GUI-exported 4-up PNG comparison grid on Cosmic Prince illustrating fine high-frequency detail retention at Q = 75 vs. heavy 8×8 blocking at Q = 10.

5.2 Quantitative Benchmark Results: Grayscale vs. Color Images

To measure quality versus estimated storage systematically on the two bundled test images, benchmarks were swept across Q ∈ {5, 10, 15, ..., 100} on assets/sample.png (256×256 grayscale, raw = 64.00 KB) and assets/sample_color.png (256×256×3 RGB, raw = 192.00 KB). Results are presented in Tables 5.1 and 5.2 using **estimated** sizes from estimate_compressed_size().

Table 5.1: Quantitative Compression Performance on Tested Grayscale Image (assets/sample.png, 256×256, Raw = 64.00 KB)

Quality Q	Est. Size (KB)	Est. CR (:1)	Est. BPP	Sparsity (% Zero)	MSE	PSNR (dB)
5	8.34	7.67×	1.043	94.26%	335.554	22.87
10	8.86	7.23×	1.107	93.86%	148.224	26.42
25	11.74	5.45×	1.468	91.61%	76.828	29.28
40	15.70	4.08×	1.963	88.51%	46.614	31.45
50	18.35	3.49×	2.293	86.45%	43.718	31.72
75	30.07	2.13×	3.759	77.29%	30.321	33.31
90	60.03	1.07×	7.504	53.88%	12.406	37.19
95	87.87	0.73×	10.983	32.14%	3.763	42.38
100	121.56	0.53×	15.195	5.82%	0.497	51.17

Table 5.2: Quantitative Compression Performance on Tested Color Image (assets/sample_color.png, 256×256×3, Raw = 192.00 KB)

Quality (Y / C)	Est. Size (KB)	Est. CR (:1)	Est. BPP	Sparsity (% Zero)	MSE	PSNR (dB)
5 / 2	10.80	17.78×	1.350	97.97%	932.398	18.43
10 / 5	12.82	14.98×	1.602	97.44%	696.753	19.70
25 / 12	23.64	8.12×	2.955	94.63%	350.685	22.68
40 / 20	30.05	6.39×	3.756	92.96%	128.349	27.05
50 / 25	32.63	5.88×	4.079	92.28%	96.833	28.27
75 / 37	41.67	4.61×	5.209	89.93%	57.277	30.55
90 / 45	60.65	3.17×	7.582	84.99%	42.639	31.83
95 / 47	82.26	2.33×	10.282	79.36%	39.556	32.16
100 / 50	139.66	1.37×	17.458	64.41%	35.301	32.65

At Q = 100, grayscale sample.png retains a nonzero MSE = 0.497 (PSNR = 51.17 dB) from coefficient rounding and uint8 truncation, while color sample_color.png retains MSE = 35.301 (32.65 dB) because Cb/Cr are still quantized at Q_chroma = 50. Furthermore, at Q = 95 and Q = 100 in Table 5.1, the 2-byte-per-nonzero heuristic exceeds the raw 64.00 KB baseline (Est. CR = 0.73× and 0.53×), demonstrating why production codecs add Huffman entropy coding when sparsity falls below 35%.

5.3 Rate–Distortion, Estimated Compression Ratio, and Sparsity Trends

Fig 5.3 shows monotonically increasing PSNR across Q for both tested images. On sample.png, PSNR crosses the 30 dB reference line near Q = 30 (30.24 dB), whereas on sample_color.png it crosses 30 dB near Q = 70 (30.11 dB). Note that sample.png and sample_color.png carry distinct spatial content and that no standalone ablation was run comparing Q_chroma = Q against Q_chroma = Q // 2 on sample_color.png. Moreover, halving Q does not simply double the quantization step when Q ≥ 50: at Q = 75, S(75) = 50 while Q_chroma = 37 gives S(37) = 135.1 (2.70×), and at Q = 90, S(90) = 20 while Q_chroma = 45 gives S(45) = 111.1 (5.56×).

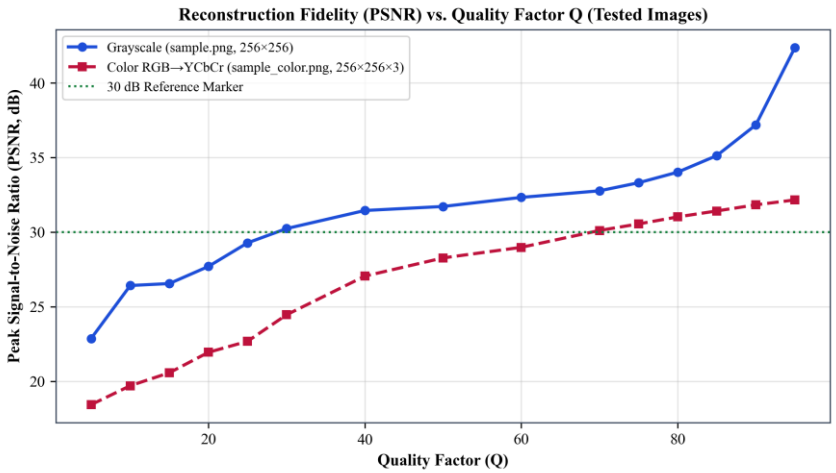


Fig 5.3: Measured PSNR (dB) vs. Quality Factor Q = 5 to 95 on the two tested images with a 30 dB reference line.

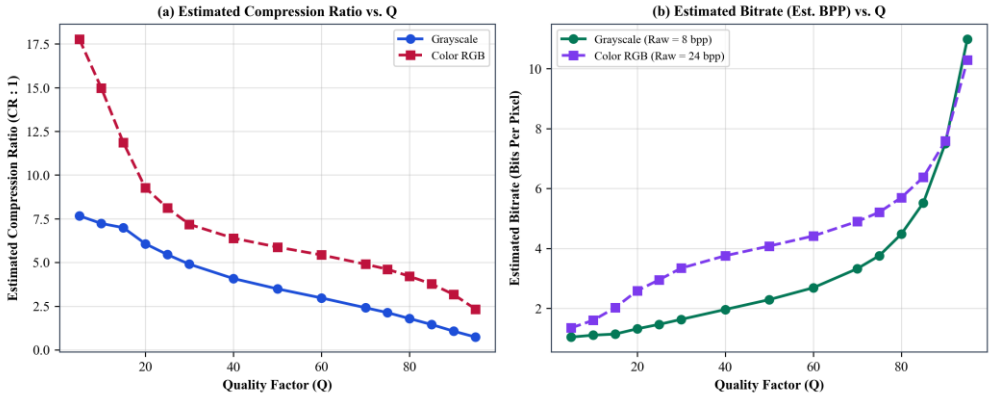


Fig 5.4: (a) Estimated Compression Ratio (Est. CR) and (b) Estimated Bitrate (Est. BPP) across Quality Factors Q on the tested images.

Figs 5.4 and 5.5 explain the estimated bitrate reductions on these tested images: between Q = 5 and Q = 75, quantized sparsity remains high (77.29%–94.26% zeros on grayscale sample.png; 89.93%–97.97% average zeros across RGB channels on sample_color.png), producing estimated compression ratios up to 7.67× and 17.78×, respectively.

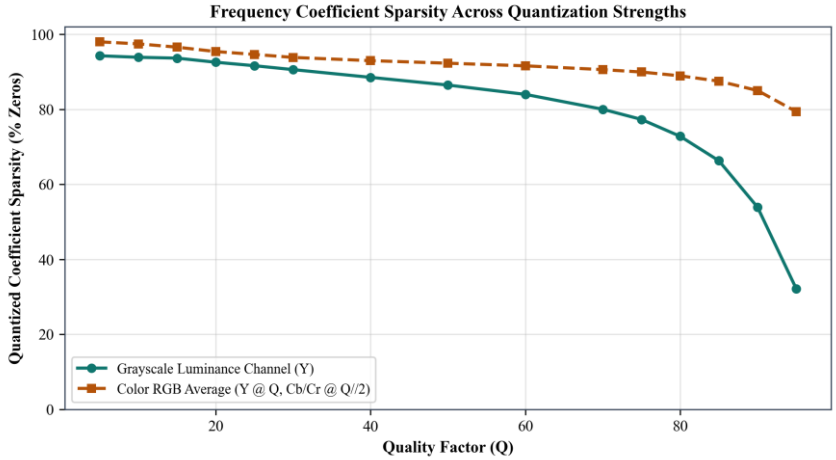


Fig 5.5: Percentage of quantized 8×8 DCT coefficients coerced to exact zeros across Quality Factors Q = 5 to 95 on the tested images.

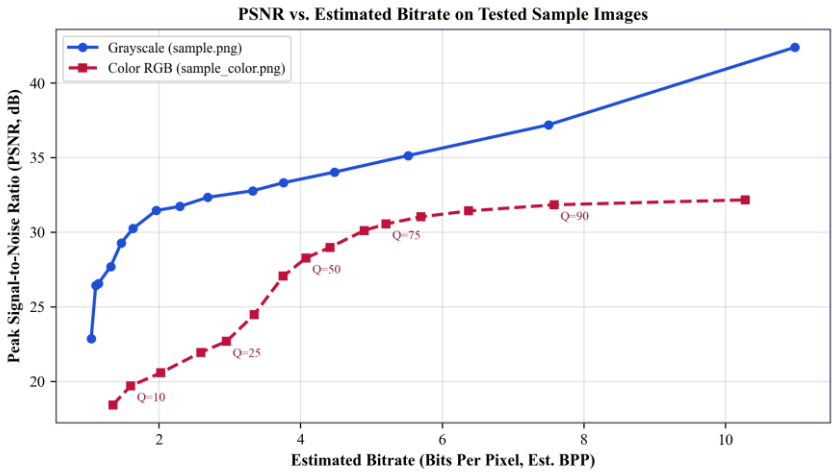


Fig 5.6: PSNR vs. Estimated Bitrate (Est. BPP) curve on the two tested sample images showing diminishing returns above Q = 75.

Plotting PSNR against Estimated BPP in Fig 5.6 reveals diminishing returns at high quality on the tested color sample: raising Q from 75 (5.209 Est. BPP, 30.55 dB) to 95 (10.282 Est. BPP, 32.16 dB) nearly doubles estimated bitrate while adding only 1.61 dB of PSNR. Thus, Q = 50–75 serves as a practical rate–distortion trade-off observed in this experiment rather than a universal optimum across arbitrary imagery.

5.4 Bitstream Estimate vs. File-Container Storage Discrepancy Analysis

A frequent point of confusion during image compression experiments is why the GUI reports Original: 192.0 KB even for a small JPG file on disk, or why saving a compressed reconstruction as .png yields a larger disk file than the reported Compressed: estimate. Table 5.3 distinguishes our heuristic estimate from actual encoded disk containers on sample_color.png (256×256×3).

Table 5.3: Comparison of Raw Memory, Uncompressed BMP, Heuristic Byte Estimate, PNG, and JPEG Container Sizes (sample_color.png)

Quality Q	Raw Memory (KB)	BMP Disk Size (KB)	Est. Size (KB)	Saved PNG Container (KB)	Saved JPG (q=100) (KB)	PSNR (dB)
10	192.00	192.05	12.82	16.07	31.24	19.70
50	192.00	192.05	32.63	41.68	33.21	28.27
75	192.00	192.05	41.67	78.54	40.73	30.55
90	192.00	192.05	60.65	96.12	47.71	31.83

First, disk files in .jpg or .png are already compressed containers; once decoded into RAM for display and matrix processing, a 256×256 RGB image unconditionally expands to 256 × 256 × 3 = 196,608 bytes (192.00 KB), matching a Windows BMP file (192.05 KB including its 54-byte header). Benchmarking against raw pixel memory isolates the DCT pipeline.

Second, saving reconstructed pixels to .png at Q = 75 writes 78.54 KB—larger than the 41.67 KB heuristic estimate—because lossless PNG DEFLATE must preserve every quantization rounding residual and sharp 8×8 tile boundary exactly. Conversely, saving as .jpg (quality=100, 40.73 KB) executes a **second lossy JPEG encode**, altering 163,546 of 196,608 channel values relative to lossless .png.

CHAPTER 6
CONCLUSION AND FUTURE SCOPE

6.1 Conclusion

This project successfully designed, implemented, and verified a transparent from-scratch 2-D DCT image compression and quality analysis pipeline for Digital Image Processing education. Every stage of the chain—BT.601 color transformation with +128 offset convention, -128 channel shifting, reflection-padded 8×8 tiling, separable orthonormal DCT-II/IDCT-III, JPEG-style luminance quantization scaling, and heuristic byte estimation—was implemented clearly in NumPy.

Experimental runs verified strong energy compaction (86.04% DC energy on the Gonzalez & Woods test block in the Theory Dialog) and 77.29%–94.26% coefficient sparsity for $Q \in [5, 75]$ on tested sample.png. Across the two bundled benchmark images, the pipeline reported estimated compression ratios of 2.13×–7.23× (grayscale) and 4.61×–14.98× (color) across Q = 10–75, achieving a practical visual-fidelity versus estimated-bitrate trade-off in the Q = 50–75 range.

Paired with an interactive Tkinter desktop GUI, dual PNG/JPG container export, a 5-tab Flowchart and Numerical Calculation pop-up dialog, a scriptable CLI, and 32 passing automated tests, the system serves effectively as both a practical experimental tool and a viva presentation laboratory.

6.2 Limitations of the Current Implementation

Several deliberate boundaries define the current educational scope. First, estimate_compressed_size() only estimates byte cost using 2 bytes per nonzero quantized coefficient plus 1 byte per block (EOB) rather than writing an encoded variable-length Huffman bitstream (Est. CR drops below 1.0× at Q = 95–100). Second, the main pipeline truncates float reconstructions via .astype(np.uint8) (32.54 dB vs. 32.62 dB rounded in explanation.py), chroma planes lack 4:2:0 spatial subsampling, and color benchmarking was limited to bundled sample imagery without a Q_chroma ablation study.

6.3 Future Scope

- **Canonical Huffman Bitstream Writer:** Implement JPEG baseline DC differential coding (DPCM) and variable-length Huffman entropy tables to emit valid, browser-readable .jpg binary bitstreams directly from scratch.

- **4:2:0 Chroma Subsampling Module:** Add optional 2×2 spatial averaging of Cb and Cr planes prior to DCT splitting to achieve realistic 2× extra chromatic storage savings alongside psychovisual quantization.
- **Perceptual Structural Similarity (SSIM):** Integrate Structural Similarity Index (SSIM) [14] and Multi-Scale SSIM alongside MSE and PSNR inside the live GUI readout and benchmark plots.
- **Discrete Wavelet Transform (DWT) Comparator:** Extend the educational pop-up with a Haar/Daubechies 2-D DWT compression mode [12] to let students contrast JPEG-style 8×8 blocking artifacts against wavelet blurring side-by-side.

REFERENCES

[1] Shannon, C.E. A mathematical theory of communication. Bell Syst. Tech. J. 1948, 27, 379–423.

[2] Gonzalez, R.C.; Woods, R.E. Digital Image Processing, 4th ed.; Pearson Education: New York, NY, USA, 2018.

[3] Wallace, G.K. The JPEG still picture compression standard. Commun. ACM 1991, 34, 30–44.

[4] Ahmed, N.; Natarajan, T.; Rao, K.R. Discrete cosine transform. IEEE Trans. Comput. 1974, C-23, 90–93.

[5] Huffman, D.A. A method for the construction of minimum-redundancy codes. Proc. IRE 1952, 40, 1098–1101.

[6] Rao, K.R.; Yip, P. Discrete Cosine Transform: Algorithms, Advantages, Applications; Academic Press: Boston, MA, USA, 1990.

[7] ISO/IEC 10918-1. Information Technology — Digital Compression and Coding of Continuous-Tone Still Images: Requirements and Guidelines; ISO: Geneva, Switzerland, 1994.

[8] Pennebaker, W.B.; Mitchell, J.L. JPEG: Still Image Data Compression Standard; Van Nostrand Reinhold: New York, NY, USA, 1993.

[9] Mannos, J.L.; Sakrison, D.J. The effects of a visual fidelity criterion on the encoding of images. IEEE Trans. Inf. Theory 1974, 20, 525–536.

[10] ITU-R Recommendation BT.601-7. Studio Encoding Parameters of Digital Television for Standard 4:3 and Wide-Screen 16:9 Aspect Ratios; ITU: Geneva, Switzerland, 2011.

[11] Boutell, T. PNG (Portable Network Graphics) Specification Version 1.0 (RFC 2083). W3C Recommendation / IETF, 1997.

[12] Taubman, D.S.; Marcellin, M.W. JPEG2000: Image Compression Fundamentals, Standards and Practice; Springer: New York, NY, USA, 2002.

[13] Bankoski, J.; Wilkins, P.; Xu, Y. Technical overview of VP8, an open source video codec for the web. In Proc. IEEE Int. Conf. Multimedia and Expo (ICME); Barcelona, Spain, 2011; pp. 1–6.

[14] Wang, Z.; Bovik, A.C.; Sheikh, H.R.; Simoncelli, E.P. Image quality assessment: From error visibility to structural similarity. IEEE Trans. Image Process. 2004, 13, 600–612.

[15] Huynh-Thu, Q.; Ghanbari, M. Scope of validity of PSNR in image/video quality assessment. Electron. Lett. 2008, 44, 800–801.

[16] Independent JPEG Group (IJG). libjpeg Reference Implementation and Quality Table Scaling Rules (jcparam.c); Release 6b/9e, 1998.

[17] Harris, C.R.; Millman, K.J.; van der Walt, S.J.; et al. Array programming with NumPy. Nature 2020, 585, 357–362.

[18] Clark, A. Pillow (PIL Fork) Python Imaging Library Documentation, Version 11.0; Python Software Foundation, 2024.

APPENDIX A
KEY SOURCE CODE EXCERPTS AND TEST VERIFICATION LOG

A.1 Core Mathematical Modules (dct_core/dct.py and quant.py)

Listing A.1: dct_core/dct.py — From-Scratch 8×8 Orthonormal 2-D DCT & Batch Einsum Engine

```
"""
dct_core.dct
============
From-scratch 2-D Type-II Discrete Cosine Transform (DCT) and its inverse.

Why we implement it ourselves (instead of calling scipy.fft.dctn):
  * It keeps the math visible and aligned with the syllabus.
  * The 8x8 DCT matrix is precomputed once and reused for every block,
    so the cost on a 512x512 image is essentially 4096 tiny 8x8 matrix products.
  * No external FFT dependency to demo live.

The forward 2-D DCT of an 8x8 block x is:
    X = D @ x @ D.T
and the matching inverse (Type-III) is:
    x = D.T @ X @ D
where D is the 8x8 matrix whose (k, i) entry is:
    D[k, i] = alpha(k) * cos((2i + 1) * k * pi / 16)
with   alpha(0)     = 1/sqrt(8)
       alpha(k > 0) = sqrt(2/8) = 1/2.

With this normalization D is orthonormal, so D @ D.T == I and
D.T @ D == I. The inverse above is therefore the exact transpose.
"""

from __future__ import annotations

import numpy as np

BLOCK: int = 8


def _build_dct_matrix(n: int = 8, dtype=np.float64) -> np.ndarray:
    k = np.arange(n, dtype=dtype).reshape(n, 1)
    i = np.arange(n, dtype=dtype).reshape(1, n)
    alpha = np.where(k == 0, 1.0 / np.sqrt(n), np.sqrt(2.0 / n))
    return alpha * np.cos((2 * i + 1) * k * np.pi / (2 * n))


D: np.ndarray = _build_dct_matrix(BLOCK, dtype=np.float64)


def dct2(block: np.ndarray) -> np.ndarray:
    """Forward 2-D DCT-II of a single 8x8 block.

    Input shape (8, 8), float-like. Returns (8, 8) float64 coefficients.
    """
    if block.shape != (BLOCK, BLOCK):
        raise ValueError(f"dct2 expects an {(BLOCK, BLOCK)} block, got {block.shape}")
    x = np.asarray(block, dtype=np.float64)
    return D @ x @ D.T


def idct2(coeffs: np.ndarray) -> np.ndarray:
    """Inverse 2-D DCT-III (matching dct2) of a single 8x8 coefficient block."""
    if coeffs.shape != (BLOCK, BLOCK):
```

```
        raise ValueError(f"idct2 expects an {(BLOCK, BLOCK)} block, got {coeffs.shape}")
    X = np.asarray(coeffs, dtype=np.float64)
    return D.T @ X @ D


def dct2_batch(blocks: np.ndarray) -> np.ndarray:
    """Forward DCT applied to a stack of N blocks of shape (N, 8, 8).

    Computes  D @ x @ D.T   for every block.
    """
    if blocks.ndim != 3 or blocks.shape[1:] != (BLOCK, BLOCK):
        raise ValueError(f"dct2_batch expects (N, {BLOCK}, {BLOCK}), got {blocks.shape}")
    x = np.asarray(blocks, dtype=np.float64)
    # (n, i, l) = sum_{j, k} D[i, j] * x[n, j, k] * D.T[k, l]
    return np.einsum("ij,njk,kl->nil", D, x, D.T)


def idct2_batch(coeffs: np.ndarray) -> np.ndarray:
    """Inverse DCT applied to a stack of N blocks of shape (N, 8, 8).

    Computes  D.T @ X @ D   for every block.
    """
    if coeffs.ndim != 3 or coeffs.shape[1:] != (BLOCK, BLOCK):
        raise ValueError(f"idct2_batch expects (N, {BLOCK}, {BLOCK}), got {coeffs.shape}")
    X = np.asarray(coeffs, dtype=np.float64)
    # (n, i, l) = sum_{j, k} D.T[i, j] * X[n, j, k] * D[k, l]
    return np.einsum("ij,njk,kl->nil", D.T, X, D)
```

Listing A.2: dct_core/quant.py — Base Luminance Table, Quality Scaling & Quantization

```
"""
dct_core.quant
==============
JPEG-style quantization for DCT coefficients.

We hard-code the standard luminance table used by JPEG at quality factor 50
and the standard scaling rule so the same table can be adapted to any Q
between 1 and 100. Higher Q = finer quantization = bigger file = better quality.

This is the heart of why DCT compression is *lossy*: dividing by the
quantization table and rounding to an integer throws away small
high-frequency details that the eye barely notices.
"""

from __future__ import annotations

import numpy as np


LUM_TABLE_Q50: np.ndarray = np.array(
    [
        [16, 11, 10, 16, 24, 40, 51, 61],
        [12, 12, 14, 19, 26, 58, 60, 55],
        [14, 13, 16, 24, 40, 57, 69, 56],
        [14, 17, 22, 29, 51, 87, 80, 62],
        [18, 22, 37, 56, 68, 103, 77, 62],
        [30, 35, 55, 64, 81, 104, 92, 75],
        [50, 72, 78, 87, 103, 121, 100, 78],
        [68, 92, 95, 98, 112, 100, 103, 99],
    ],
    dtype=np.int32,
)


def scale_table(base: np.ndarray, q: int) -> np.ndarray:
    """Scale a base quantization table for an arbitrary JPEG Q-factor in [1, 100].
```

```
    The rule is the standard one used by libjpeg:
        if Q < 50:  scale = 5000 / Q
        else:       scale = 200 - 2 * Q
        scaled[i]  = clip(floor((base[i] * scale + 50) / 100), 1, 255)
    """
    if not 1 <= q <= 100:
        raise ValueError(f"Quality factor Q must be in [1, 100], got {q}")
    scale = 5000 / q if q < 50 else 200 - 2 * q
    scaled = np.floor((base.astype(np.float64) * scale + 50.0) / 100.0)
    return np.clip(scaled, 1, 255).astype(np.int32)


def quantize(coeffs: np.ndarray, table: np.ndarray) -> np.ndarray:
    """Quantize DCT coefficients: round(C / table). Accepts (8, 8) or (N, 8, 8)."""
    if coeffs.shape[-2:] != table.shape:
        raise ValueError(
            f"quantize: coefficient shape {coeffs.shape} does not match table
{table.shape}"
        )
    return np.round(coeffs.astype(np.float64) / table.astype(np.float64)).astype(np.int32)


def dequantize(qcoeffs: np.ndarray, table: np.ndarray) -> np.ndarray:
    """Dequantize: qcoeffs * table (recovers approximate DCT coefficients)."""
    if qcoeffs.shape[-2:] != table.shape:
        raise ValueError(
            f"dequantize: qcoeff shape {qcoeffs.shape} does not match table {table.shape}"
        )
    return qcoeffs.astype(np.float64) * table.astype(np.float64)
```

A.2 Pipeline Orchestration and Automated Test Verification

Listing A.3: pipeline/pipeline.py — End-to-End Grayscale & YCbCr Compression Orchestrator

```
"""
pipeline.pipeline
=================
Orchestrates the full DCT compression pipeline for both grayscale and
colour images.

For colour images we go RGB → YCbCr → per-channel DCT/quant/IDCT
→ YCbCr → RGB. The luminance channel Y is processed at the user-selected
quality Q; the chroma channels Cb and Cr are processed at
Q_chroma = max(1, Q // 2), matching how JPEG treats chroma more
aggressively than luma.

The pipeline is vectorized: every channel is processed as one batch
of (N, 8, 8) blocks so the demo stays fast on a typical laptop.
"""

from __future__ import annotations

from dataclasses import dataclass
from typing import Optional

import numpy as np

from dct_core.blocks import merge_blocks, split_blocks
from dct_core.dct import dct2_batch, idct2_batch
from dct_core.quant import (
    LUM_TABLE_Q50,
    scale_table,
    quantize,
```

```
    dequantize,
)
from quality.metrics import (
    compression_ratio,
    estimate_compressed_size,
    mse,
    psnr,
)


@dataclass
class CompressionResult:
    reconstructed: np.ndarray           # uint8 image with the same shape as input
    cr: float                           # compression ratio (orig / compressed)
    mse_val: float                      # mean squared error over all pixels
    psnr_val: float                     # decibels, +inf if lossless
    original_bytes: int                 # raw pixel bytes (W * H * C)
    compressed_bytes: int               # zig-zag / RLE estimator result
    quality: int                        # Q used for luminance
    chroma_quality: int                 # Q used for chroma channels


def _compress_channel(channel: np.ndarray, q: int) -> tuple[np.ndarray, np.ndarray,
np.ndarray]:
    """Compress one 2-D channel (H, W) at quality Q and return (recon, qblocks, table)."""
    shifted = channel.astype(np.float64) - 128.0
    blocks, orig_shape, padded_shape = split_blocks(shifted)
    coeffs = dct2_batch(blocks)
    table = scale_table(LUM_TABLE_Q50, q)
    qblocks = quantize(coeffs, table)
    rec_coeffs = dequantize(qblocks, table)
    rec_shifted = idct2_batch(rec_coeffs)
    rec = merge_blocks(rec_shifted, orig_shape, padded_shape) + 128.0
    return rec, qblocks, table


def compress_image(image_u8: np.ndarray, quality: int = 75) -> CompressionResult:
    """Top-level entry: accept (H, W) grayscale or (H, W, 3) RGB uint8."""
    if not (1 <= quality <= 100):
        raise ValueError(f"quality must be in [1, 100], got {quality}")
    if image_u8.dtype != np.uint8:
        image_u8 = image_u8.astype(np.uint8)

    if image_u8.ndim == 2:
        rec_f, qblocks, _ = _compress_channel(image_u8, quality)
        rec_u8 = np.clip(rec_f, 0, 255).astype(np.uint8)
        original_bytes = int(image_u8.size)
        compressed_bytes = estimate_compressed_size(qblocks)
        cr = compression_ratio(original_bytes, compressed_bytes)
        m = mse(image_u8.astype(np.float64), rec_u8.astype(np.float64))
        p = psnr(image_u8.astype(np.float64), rec_u8.astype(np.float64))
        return CompressionResult(rec_u8, cr, m, p, original_bytes, compressed_bytes,
                                 quality, quality)

    if image_u8.ndim == 3 and image_u8.shape[-1] == 3:
        from .color import rgb_to_ycbcr, ycbcr_to_rgb
        ycbcr = rgb_to_ycbcr(image_u8)
        y_q = quality
        c_q = max(1, quality // 2)
        y_rec, y_qblocks, _ = _compress_channel(ycbcr[..., 0], y_q)
        cb_rec, cb_qblocks, _ = _compress_channel(ycbcr[..., 1], c_q)
        cr_rec, cr_qblocks, _ = _compress_channel(ycbcr[..., 2], c_q)
        rec_ycbcr = np.stack([y_rec, cb_rec, cr_rec], axis=-1)
        rec_u8 = ycbcr_to_rgb(rec_ycbcr)
        original_bytes = int(image_u8.size)
        compressed_bytes = (
            estimate_compressed_size(y_qblocks)
            + estimate_compressed_size(cb_qblocks)
```

```
            + estimate_compressed_size(cr_qblocks)
        )
        cr = compression_ratio(original_bytes, compressed_bytes)
        m = mse(image_u8.astype(np.float64), rec_u8.astype(np.float64))
        p = psnr(image_u8.astype(np.float64), rec_u8.astype(np.float64))
        return CompressionResult(rec_u8, cr, m, p, original_bytes, compressed_bytes,
                                 y_q, c_q)

    raise ValueError(f"compress_image expects (H, W) or (H, W, 3), got {image_u8.shape}")
```

Listing A.4: Automated Verification Output (python -m pytest tests/ -v)

```
$ python -m pytest tests/ -v
platform win32 -- Python 3.13.7, pytest-8.3.4, pluggy-1.7.0
collected 32 items

tests/test_color.py::test_roundtrip_rgb PASSED                           [  3%]
tests/test_color.py::test_compress_color_image_runs PASSED               [  6%]
tests/test_color.py::test_color_size_includes_three_channels PASSED      [  9%]
tests/test_dct.py::test_orthogonality PASSED                             [ 12%]
tests/test_dct.py::test_roundtrip_single_block PASSED                    [ 15%]
tests/test_dct.py::test_roundtrip_batch PASSED                           [ 18%]
tests/test_dct.py::test_constant_block_dc_only PASSED                    [ 21%]
tests/test_dct.py::test_input_shape_errors PASSED                        [ 25%]
tests/test_explanation.py::test_calculate_block_steps_standard_sample PASSED [ 28%]
tests/test_explanation.py::test_quality_factor_scaling_extremes PASSED   [ 31%]
tests/test_explanation.py::test_format_matrix PASSED                     [ 34%]
tests/test_explanation.py::test_text_generators PASSED                   [ 37%]
tests/test_explanation.py::test_full_image_metrics_breakdown PASSED      [ 40%]
tests/test_explanation.py::test_report_generation PASSED                 [ 43%]
tests/test_explanation.py::test_cli_explain_and_report_flags PASSED      [ 46%]
tests/test_explanation.py::test_gui_theory_popup_dialog_lifecycle PASSED [ 50%]
tests/test_io.py::test_roundtrip_png_grayscale PASSED                    [ 53%]
tests/test_io.py::test_roundtrip_png_color PASSED                        [ 56%]
tests/test_io.py::test_comparison_grid_writes_file PASSED                [ 59%]
tests/test_metrics.py::test_mse_zero_for_identical PASSED                [ 62%]
tests/test_metrics.py::test_psnr_inf_for_identical PASSED                [ 65%]
tests/test_metrics.py::test_psnr_known_noise PASSED                      [ 68%]
tests/test_metrics.py::test_compression_ratio_basic PASSED               [ 71%]
tests/test_metrics.py::test_estimate_size_responds_to_q PASSED           [ 75%]
tests/test_pipeline.py::test_pipeline_smooth_image_high_psnr PASSED      [ 78%]
tests/test_pipeline.py::test_pipeline_non_multiple_of_8_dimensions PASSED [ 81%]
tests/test_pipeline.py::test_q100_is_lossless_within_one PASSED          [ 84%]
tests/test_quant.py::test_q100_means_unit_step PASSED                    [ 87%]
tests/test_quant.py::test_q1_is_aggressive PASSED                        [ 90%]
tests/test_quant.py::test_q50_is_base_table PASSED                       [ 93%]
tests/test_quant.py::test_quantize_dequantize_error_bound PASSED         [ 96%]
tests/test_quant.py::test_invalid_q_raises PASSED                        [100%]
============================== 32 passed in 16.86s ==============================
```